

CH 2.3: QUERY PROCESSING

Contents

Query Processing	1
Characteristics of Query Processors	1
Layers of Query Processing	2
Challenges in Distributed Environments.....	3
1. Query decomposition	3
A. Normalization	4
B. Analysis	5
C. Elimination of Redundancy	7
D. Rewriting	7
2. Data Localization	8
Translating a Global Query	8
3. Global Query Optimization	10
4. Local Optimization.....	10
Query Processing vs Query Optimization.....	11

Query Processing

Query Processing refers to the steps involved in transforming a high-level query (e.g., in SQL) into an efficient execution plan and then executing it to retrieve the required results. In a **Distributed DBMS**, query processing becomes more complex due to the distribution of data across multiple sites.

Purpose and Objectives of Query Processing

- **Correctness:** The result should be the same as if the query were executed on a centralized system.
- **Efficiency:** Reduce response time, communication cost, and local processing cost.
- **Resource Optimization:** Efficiently use CPU, memory, and network bandwidth.
- **Scalability:** Performance should not degrade significantly as the system grows.
- **Parallelism:** Exploit parallel execution to speed up processing.

Characteristics of Query Processors

A **Query Processor** is a component of a DBMS responsible for interpreting and executing database queries. In a DDBMS, its key characteristics are:

- **Data Location Awareness:** It must know the location of data fragments across different sites.
- **Communication Management:** Handles inter-site communication to gather data.
- **Decomposition:** Breaks global queries into sub-queries suitable for local sites.
- **Optimization Capability:** Optimizes the query execution considering data distribution, replication, and network cost.
- **Heterogeneity Handling:** May need to work with different DBMSs or schemas at various sites.
- **Fault Tolerance:** Manages failures of sites or links during processing.

Layers of Query Processing

The process of query execution in a DDBMS can be divided into several layers:

a. Query Decomposition

- Translates the high-level query into an internal form.
- Checks for syntactic and semantic correctness.
- Converts query into relational algebra or calculus expressions.

b. Data Localization

- Identifies which fragments of the distributed data are relevant.
- Rewrites the query using fragment definitions.
- Resolves fragment locations (horizontal, vertical, derived fragmentation).

c. Global Optimization

- Considers various execution strategies across sites.
- Estimates costs (I/O, CPU, communication).
- Chooses an optimal plan based on cost models.

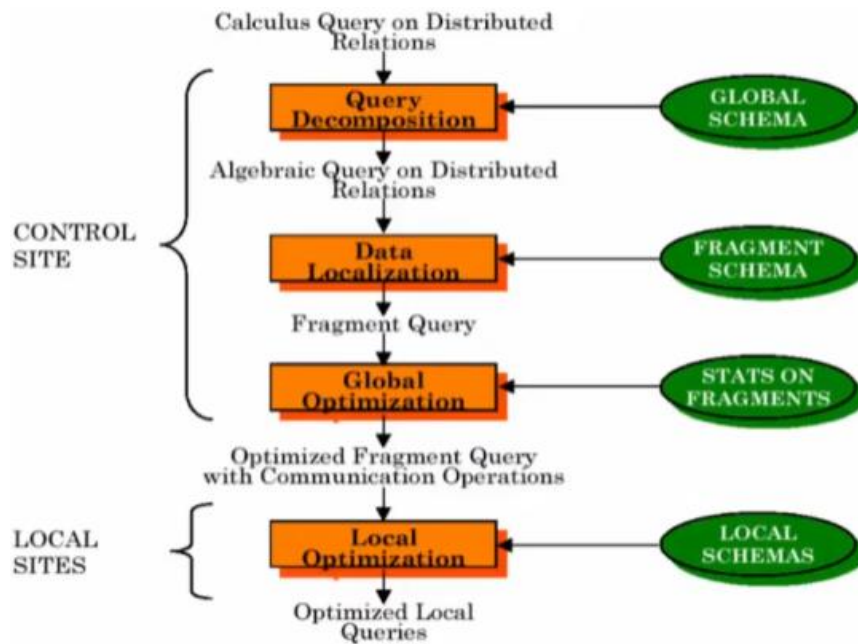
d. Local Optimization

- Optimizes subqueries locally at each site using local DBMS capabilities.
- Applies local indexes, join orders, etc.

e. Execution

- Executes the plan in a coordinated manner.
- Collects intermediate results.
- Combines and presents the final output to the user.

Layer	Input	Process (Work Done)	Output
1. Query Decomposition	- High-level user query (e.g., SQL) - Global schema and integrity constraints	- Parses and checks for syntax & semantics - Translates query into relational algebra - Verifies correctness	- Canonical relational algebra query
2. Data Localization	- Canonical relational algebra query - Fragmentation schema - Fragment location info	- Rewrites query using fragment definitions - Maps subqueries to data fragments at sites	- Fragmented query with site-specific subqueries
3. Global Query Optimization	- Fragmented query - Data distribution metadata - Cost models (CPU, I/O, communication)	- Generates alternative execution strategies - Estimates costs - Chooses most efficient global plan	- Optimized global query execution plan
4. Local Optimization	- Subqueries for each site - Local DB schema and stats	- Applies local DBMS optimization (e.g., join ordering, index usage) - Refines physical query plans	- Optimized subquery execution plans for each site
5. Query Execution	- Local and global execution plans - Actual distributed data	- Executes subqueries at respective sites - Manages inter-site communication - Merges intermediate results	- Final result to the user



Challenges in Distributed Environments

Query processing in distributed databases faces several unique challenges:

- **Data Fragmentation and Replication:** Managing and locating multiple fragments or replicas.
- **Network Latency and Bandwidth:** Communication costs can dominate execution time.
- **Site Autonomy:** Sites may be independent and may not cooperate fully.
- **Heterogeneity:** Different DBMSs, data models, and query languages.
- **Dynamic Load Balancing:** Need to distribute workload efficiently across nodes.
- **Fault Tolerance:** Ensuring query completion despite site/link failures.
- **Security and Privacy:** Ensuring safe data exchange across potentially insecure links.

1. Query decomposition

Query decomposition is the first stage of query processing where a high-level calculus query (e.g., SQL) is transformed into an intermediate, simplified, and logically equivalent form—usually relational algebra (with algebraic operations like select, join, rename etc).

It ensures that the query is syntactically and semantically correct before any distributed optimization or execution occurs.

Why Query Decomposition?

- Improves performance by simplifying and optimizing before distribution.
- Helps in **localizing** subqueries to specific sites.
- Enables **parallelism** and **site autonomy** in execution.

Steps of Query Decomposition

1. Normalization

- Transforms a complex query into a simpler, standardized form.
- E.g., converting nested queries into flat relational algebra expressions.
- Goal: Simplify the query structure.

2. Analysis

- Checks the **query's syntactic and semantic correctness**.
- Ensures that attributes exist, types match, etc.
- Also checks for **query equivalence** to maintain correctness.

3. Simplification / Elimination of Redundancy

- Removes unnecessary operations like redundant joins, selections, or projections.
- Example: $\text{SELECT } * \text{ WHERE age} > 25 \text{ AND age} > 20 \rightarrow \text{eliminate age} > 20$.

4. Restructuring / Canonical Transformation

- Converts the query into an **equivalent but more efficient** form.
- May involve using **heuristic rules** like pushing selections down or combining joins.

A. Normalization

① It is the goal of normalization to transform the query to a normal form to facilitate processing.

② It is the 1st stage of Query decomposition, where the SQL query is converted to relational algebra exp.

③ Simplifies nested queries, subqueries and complex expr into basic operations (like selection, projection, join etc).

④ There are 2 normal forms it is converted to —

→ Conjunctive NF $[(p_1 \wedge p_2 \dots p_n) \wedge \dots \wedge (p_x \wedge p_y \dots p_m)]$

→ Disjunctive NF $[(p_1 \wedge p_2 \dots p_n) \vee \dots \vee (p_x \wedge p_y \dots p_m)]$

Example

Select Ename From EMP, ASG

Where EMP.Eno = ASG.Eno AND ASG.Pro = "P1"
AND DUR = 12 OR DUR = 24

Ans) CNF

$\text{EMP.Eno} = \text{ASG.Eno} \wedge \text{ASG.Pro} = "P1" \wedge (\text{DUR} = 12 \vee \text{DUR} = 24)$

DNF

$(\text{EMP.Eno} = \text{ASG.Eno} \wedge \text{ASG.Pro} = "P1" \wedge \text{DUR} = 12) \vee$
 $(\text{EMP.Eno} = \text{ASG.Eno} \wedge \text{ASG.Pro} = "P1" \wedge \text{DUR} = 24)$

Any of the above may be selected but, DNF contain redundant subexpressions. We select CNF.

$\therefore \Pi_{\text{Ename}} (\sigma_{\text{EMP.Eno} = \text{ASG.Eno} \wedge \text{ASG.Pro} = "P1" \wedge (\text{DUR} = 12 \vee \text{DUR} = 24)} (\text{EMP} \times \text{ASG}))$

B. Analysis

Identify and reject type incorrect and semantically incorrect queries

2. **Type Incorrect Queries:** A type incorrect query refers to a query where data types are used improperly or inconsistently, making the operation invalid or meaningless. Occurs When:

- Operations are applied to incompatible data types.
- Functions expecting specific types are given wrong ones.

Examples:

a. Comparing incompatible types: salary is numeric, but 'high' is a string → type mismatch.

```
SELECT * FROM Employee WHERE salary = 'high';
```

b. Performing arithmetic on non-numeric fields: name is a string; addition with a number is type incorrect.

```
SELECT name + 100 FROM Student;
```

c. Using aggregate functions on wrong types: AVG() expects numeric values, but name is a string.

```
SELECT AVG(name) FROM Student;
```

3. **Semantically Incorrect Queries:** A semantically incorrect query is one that is syntactically and type-wise valid, but does not make logical sense in the context of the schema or database semantics.

Occurs When:

- The query references nonexistent attributes or relations.
- Operations are attempted on invalid structures.
- Query logic contradicts the schema's meaning or constraints.

Examples:

a. Referencing nonexistent attribute: roll_no is not part of the Course table → semantically invalid.

```
SELECT roll_no FROM Course;
```

b. Invalid join condition: This is syntactically correct but logically makes no sense, unless name and title are meant to be equal, which is unlikely.

```
SELECT * FROM Student, Course WHERE Student.name = Course.title;
```

c. Using a relation that doesn't exist: If Marks is not a defined table, the query is semantically incorrect.

```
SELECT * FROM Marks;
```

d. Checking access control (if the user lacks permission): Table exists, but user doesn't have access → semantically invalid from a system perspective.

```
SELECT * FROM Salary;
```

We use different data structures to detect the semantically incorrect queries:

1. Connection Graph (or Query Graph)

- **Nodes** represent **relations (tables)**.
- **Edges** represent **join conditions** between the relations if both connected nodes represent a operand relation, else it is **projection**.
- To represent the **structure of a query**, especially **relationships among relations (tables)** involved in a query.
- Used to detect **missing joins, invalid references, or disconnected relations**, which indicate **semantic errors**.

2. Join Graph

- It is a subgraph of the query graph that represents only the **join conditions** among relations.
- If the query graph is connected the query is semantically correct else not.
- Nodes:** relations involved in the join
- Edges:** join predicates, typically equi-joins on foreign key relationships

Example of analysis

Consider the following 2 queries. Using query graph and join graph state if they are semantically correct.

QUERY 1

```
SELECT ENAME, RESP
FROM EMP, ASG, PROJ
WHERE EMP.ENO = ASG.ENO
      AND ASG.PNO = PROJ.PNO
      AND PNAME = 'CAD/CAM'
      AND DUR >= 36
      AND TITLE = 'Programmer';
```

QUERY 2

```
SELECT ENAME, RESP
FROM EMP, ASG, PROJ
WHERE EMP.ENO = ASG.ENO
      AND PNAME = 'CAD/CAM'
      AND DUR >= 36
      AND TITLE = 'Programmer';
```

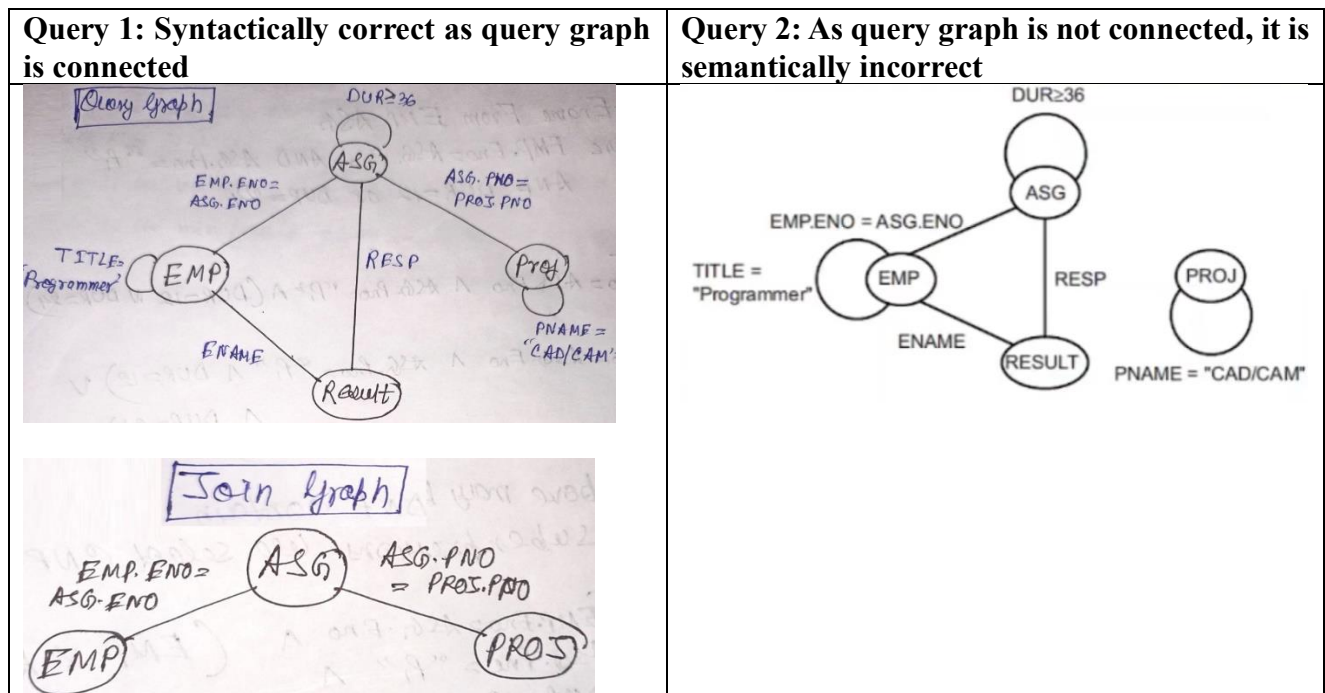
ENO	ENAME	TITLE
E1	J. Doe	Elect. Eng
E2	M. Smith	Syst. Anal.
E3	A. Lee	Mech. Eng.
E4	J. Miller	Programmer
E5	B. Casey	Syst. Anal.
E6	L. Chu	Elect. Eng.
E7	R. Davis	Mech. Eng.
E8	J. Jones	Syst. Anal.

ENO	PNO	RESP	DUR
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36
E8	P3	Manager	40

PNO	PNAME	BUDGET
P1	Instrumentation	150000
P2	Database Develop.	135000
P3	CAD/CAM	250000
P4	Maintenance	310000

TITLE	SAL
Elect. Eng.	40000
Syst. Anal.	34000
Mech. Eng.	27000
Programmer	24000

Answer



C. Elimination of Redundancy

Elimination of redundancy is the step where the system removes unnecessary or logically repeated parts of a query to make it simpler, more efficient, and semantically correct.

Example of redundancy

```
SELECT TITLE FROM EMP
WHERE EMP.ENAME = "J. Doe"
      OR (NOT(EMP.TITLE = "Programmer")
          AND (EMP.TITLE = "Elect. Eng." OR EMP.TITLE = "Programmer")
          AND NOT(EMP.TITLE = "Elect. Eng."))
```

Answer

The above query can be simplified by considering this method

- p1: TITLE = "Programmer"
- p2: TITLE = "Elect. Eng."
- p3: ENAME = "J. Doe"

Handwritten logical simplification of the query:

$$\begin{aligned} & p_3 \vee (\neg p_1 \wedge (p_2 \vee p_1) \wedge \neg p_2) \\ &= p_3 \vee ((\neg p_1 \wedge p_2) \vee (\neg p_1 \wedge p_1)) \wedge \neg p_2 \\ &= p_3 \vee ((\neg p_1 \wedge p_2 \wedge \neg p_2) \vee (\neg p_1 \wedge p_1 \wedge \neg p_2)) \\ &= p_3 \vee \text{false} \vee \text{false} \\ &= p_3 \end{aligned}$$

∴ Simplified query: Select Title from EMP
where Ename = "J. Doe"

D. Rewriting

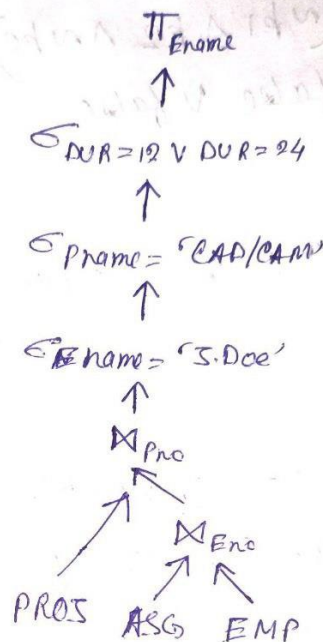
- **Rewriting** is the **final step** of query decomposition in Distributed DBMS.
- In this step, the original high-level query (usually written in relational calculus or SQL) is **rewritten into relational algebra**.
- This algebraic expression is then represented **graphically using an operator tree**.
- **Operator tree:** is a hierarchical representation where:
 - **Leaves** represent base relations (tables).
 - **Intermediate nodes** represent algebraic operations like **selection (σ)**, **projection (π)**, **join ($\bowtie$)**, etc.
 - The **root** represents the final output of the query.
- Multiple **equivalent relational algebra queries** can be derived from the same relational calculus (or SQL) query.
- However, not all of them are equally efficient, some **algebraic expressions are better** in terms of:
 - **Query cost**
 - **Communication overhead**
 - **Local processing time**

Query: Find the names of employees other than J. Doe who worked on CAD/CAM prj for either 12 or 24 yrs.

SQL Query

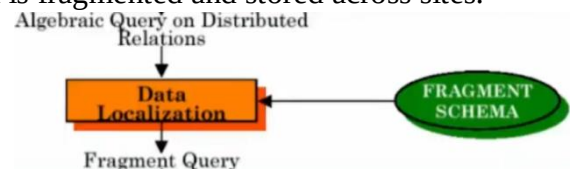
```

Select Ename
From EMP, ASG, PROJ
Where EMP.Eno=ASG.Eno
and ASG.Pno= PROJ.Pno
and EMP.Ename <> 'J. Doe'
and PRJ.PNAME = 'CAD/CAM'
and (DUR=12 V DUR=24)
  
```



2. Data Localization

- Data Localization is the process of converting a global algebraic query into queries on local data fragments in a distributed database, based on how data is fragmented and stored across sites.
- **Input:** An algebraic query on the global conceptual schema.
- **Purpose:** The main role of this layer is to localize the data involved in a query by using data distribution information from the fragment schema.
- **Explanation:**
 - In a DDBMS, relations are fragmented into disjoint subsets known as fragments each stored at different sites in distributed system.
 - The Data Localization layer determines which fragments are involved in a query and transforms the distributed query into a query of fragments.



Translating a Global Query

- In a DDBMS, a **global relation** (e.g., Student) may be **fragmented** and stored at different sites.
- Additionally, fragments might be **replicated** for efficiency and fault tolerance.
- A **user** or application issues a query on the **global schema**, without knowledge of where data is physically stored.

Example: decomposition of global query into fragments

Global Query: `SELECT * FROM Student;`

If the `Student` relation is horizontally fragmented into multiple parts:

- `Student1`
- `Student2`
- ...
- `StudentN`

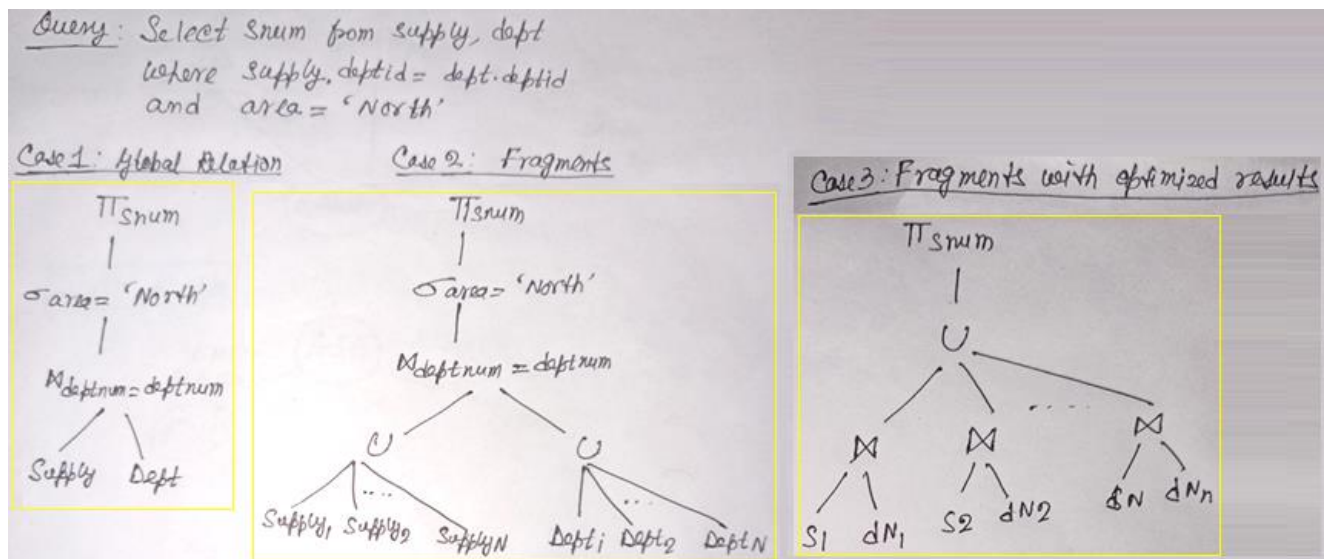
Then, the global query is decomposed into the following fragment queries:

```
SELECT * FROM Student1;  
SELECT * FROM Student2;  
...  
SELECT * FROM StudentN;
```

To execute these fragment queries, the system must know:

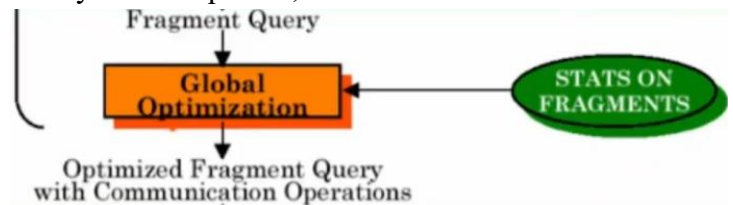
- How the `Student` relation was **fragmented**
- Whether any fragments are **replicated**
- Where each fragment or its replica is **allocated** (i.e., stored)

Example: Query tree representation



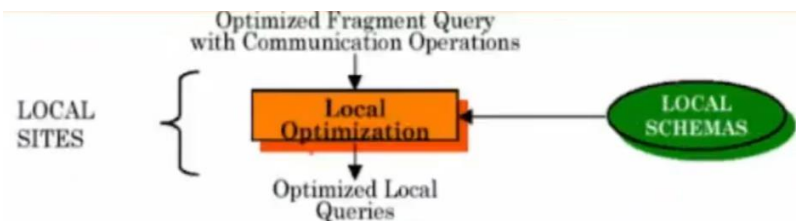
3. Global Query Optimization

- **Input:** Algebraic query on fragments (after data localization).
- **Objective:** Determine an execution strategy that minimizes overall cost (close to optimal).
- **Key Concepts:**
 - Permutation on queries leads to many similar queries, which are ordered based on cost functions, which includes:
 - Time units
 - Disk space
 - I/O operations
 - CPU usage
 - Communication cost (e.g., limited bandwidth)
 - The goal is to minimize total execution cost.
- **Factors Considered:**
 - Fragment allocation and replication
 - Fragment statistics (e.g., tuple count, selectivity)
- **Static Optimization:** cost is predicted statistically before execution, with techniques like
 - Join ordering
 - Use of semijoins to reduce data transfer
- **Output:** Optimized algebraic query with **communication operators** and **fragment references**.



4. Local Optimization

- **Performed at:** Each site that stores relevant fragments.
- **Purpose:** Optimize and execute local subqueries using the site's local schema and resources.
- **Steps:**
 - Choose appropriate algorithms for relational operators (e.g., join methods).
 - Optimize query execution plan locally.
- **Result:** Local results are computed and sent back to the query origin site for final assembly.



Query Processing vs Query Optimization

Aspect	Query Processing	Query Optimization
Definition	Entire process of executing a query from parsing to result	Subprocess focused on improving query efficiency
Scope	Broad – includes decomposition, localization, optimization, execution	Narrow – only includes finding the best execution strategy
Steps Involved	Query decomposition, data localization, global & local optimization, execution	Global and local optimization
Purpose	Get correct results from the DB	Reduce cost, time, and resources used during execution
Focus	Execution pipeline	Efficiency of execution strategy
Timing	Runs from start to end of query life cycle	Happens before actual execution
Techniques Used	Parsing, rewriting, access planning, execution	Cost-based search, heuristic rules, transformation-based rewriting
System Component	Handled by the query processor module	Handled by the query optimizer module